

DALEEN HUNNIPHER OTIENO

PROJECT PROPOSAL

Project Name:FREELANCE FLOW- One app for clients, projects, invoices, expenses

Objective: Unify client and project management in one system(MOBILE APP)

PROJECT INTRODUCTION AND BACKGROUND

By definition, remote freelancing cuts across borders. For example, a designer in Nairobi may bill a customer in London in GBP or USD while residing and paying taxes in KES thereby creating three compounding friction points. Firstly, tool fragmentation compels developers to disperse client information, project tracking, invoicing, and expense reports among many spreadsheets, applications, and notes. Second, currency conversion is done by hand and is error-prone and instead of using real-time exchange rates, it relies on ad hoc search engine conversions and mental maths. Lastly, a total divergence between project expenses and incoming cash causes tax-time issues, forcing freelancers to either overpay taxes by overlooking legitimate deductions or scramble through disorganised paperwork.

While mature platforms like FreshBooks, QuickBooks, Bonsai, and Zoho Invoice cater to freelancers, they struggle to meet the needs of solo, multi-currency remote workers on mobile devices. Desktop-first and built for general-ledger accounting, these tools are often overly complex, lock live multi-currency features behind expensive paywalls, and rely on the US or EU-centric tax models. Driven by the widespread adoption of remote work, cross-border freelancing has surged, leaving independent workers to manage the

full administrative burden of billing, currency conversion, and tax compliance without dedicated administrative support. Inadequate invoicing often leads to payment delays and disputes, while poor expense tracking forces freelancers to either miss valid tax deductions or face compliance risks. FreelanceFlow addresses this clear workflow gap through a practical, semester-scope Android application built with local-first data and key third-party integrations. By combining offline sync, external APIs, PDF generation, dynamic financial calculations, and data modeling, the project serves as a substantial software engineering exercise that delivers a complete, lightweight solution for solo remote freelancers.

METHODOLOGY

The project uses a lightweight, Agile-inspired approach structured around two-week sprints. Requirements from the backlog are tracked on a GitHub Projects Kanban board and prioritized based on milestone dependencies such as building client CRUD operations before invoicing. Rather than relying on a rigid Waterfall model, this iterative workflow de-risks complex elements like offline sync edge cases, external currency APIs, and PDF generation by delivering thin, testable vertical slices early. Each sprint concludes with a working build demonstration and a brief written retrospective to refine the build process continuously.

Development is supported by a streamlined toolchain and Git branching strategy to ensure high code quality and traceability throughout the semester. Work progresses from feature branches into a dev integration branch via pull requests, while the main branch remains stable and buildable, with tagged releases marking key project milestones. Automated testing and linting run via GitHub Actions, enforcing a strict Definition of Done. A feature is only complete when it passes all automated checks, includes unit tests for core financial and business logic, operates correctly offline, and undergoes manual testing on an Android emulator or physical device.

The project will be executed over a 1.5-month semester plan organized into seven distinct two-week sprints, moving sequentially from initial requirements and design in Sprint 0 through core features;client management, project and time tracking, currency integration, invoice generation, and expense tax estimation to final testing and polish in Sprint 6. To

deliver this functionality, the system relies on a modern, robust tech stack featuring a React, TypeScript, and Tailwind CSS frontend paired with a Node.js, Express, and Docker backend. Data management and caching are handled via PostgreSQL on AWS RDS and Redis on AWS ElastiCache, while the broader cloud infrastructure leverages AWS ECS, Lambda, S3, and CloudFront, supported by GitHub Actions for CI/CD alongside Datadog and Sentry for real-time monitoring and error tracking.

FUNCTIONAL AND NONFUNCTIONAL REQUIREMENTS

The system's functional requirements span ten core operational modules designed to deliver a complete administrative workflow for remote freelancers. Users can set up accounts with base currencies and tax profiles, manage clients and projects with multi-currency support, track time through manual entries or live timers, and log categorised expenses with receipt attachments and deductible tags. The invoicing engine automatically pulls unbilled hours, assigns unique sequential invoice numbers, fetches real-time exchange rates with visible timestamps, generates exportable professional PDFs, and tracks lifecycle statuses like sent, paid, or overdue. To complete the administrative suite, the platform calculates estimated tax deductions based on user-configured rules, displays an aggregated dashboard converting overall earnings and expenses into the home currency, queues changes for background cloud synchronization while operating fully offline using cached exchange rates, and issues local push notifications for overdue payments.

Complementing these features, the non-functional requirements prioritize speed, security, and architectural robustness to ensure a reliable user experience. Performance and offline-first standards dictate that core screens load from local caches in under one second on mid-range devices, keeping local databases as the absolute source of truth without requiring active network connectivity for primary CRUD tasks. Security and reliability are maintained through local data encryption at rest, HTTPS network communications, Firebase security rules enforcing authenticated ownership, and fault-tolerant sync operations that prevent data loss during network interruptions. Furthermore, the

application adheres to Material 3 usability and accessibility standards offering dark mode support, system font scaling, and optimized navigation flows accessible within five taps while employing a maintainable Clean Architecture with an MVVM pattern to isolate core business logic for easy unit testing and future multi-platform portability.